

FPS, dt

Animación: Movimiento discreto en Raylib

HCI

April 2026

Objetivo

Comprender con claridad:

- Qué son los FPS.
- Qué es dt .
- Qué ocurre al mover objetos sin dt .
- Qué ocurre al mover objetos con dt .

Contexto del problema

Queremos mover un círculo sobre el eje x .

La posición horizontal del círculo será:

$$x(t)$$

donde:

- $x(t)$: posición horizontal en el tiempo t .
- t : tiempo medido en segundos.
- v : velocidad horizontal.

¿Qué son los FPS?

FPS significa:

Frames Per Second

Es decir:

Cuadros por segundo

Si un programa corre a 60 FPS, intenta dibujar:

60 imágenes cada segundo

En animación, cada imagen se llama **frame**.

FPS y tiempo entre frames

Relación ideal:

$$dt \approx \frac{1}{\text{FPS}}$$

Ejemplos:

$$30 \text{ FPS} \Rightarrow dt \approx 0,0333$$

$$60 \text{ FPS} \Rightarrow dt \approx 0,0167$$

$$120 \text{ FPS} \Rightarrow dt \approx 0,0083$$

A más FPS, menor duración de cada frame.

¿Qué es dt ?

En simulación, dt significa:

$$dt = \Delta t$$

Es el tiempo transcurrido entre dos frames consecutivos.

En Raylib se obtiene con:

```
GetFrameTime()
```

Si el frame tarda mucho, dt aumenta. Si el frame tarda poco, dt disminuye.

dt como puente con el tiempo real

El computador actualiza la escena cuadro a cuadro.

Pero el movimiento físico depende del tiempo real.

Por eso usamos:

$$\Delta x = v \cdot dt$$

donde:

- Δx : desplazamiento en un frame.
- v : velocidad en píxeles por segundo.
- dt : segundos transcurridos en ese frame.

Modelo matemático del movimiento

El modelo continuo de velocidad constante es:

$$\frac{dx}{dt} = v$$

Esto significa:

- La posición cambia con el tiempo.
- La velocidad v mide qué tan rápido cambia x .

Si $v = 200$, el objeto debe avanzar:

200 píxeles en un segundo

Actualización correcta

Para simular el movimiento, usamos:

$$x_{\text{nuevo}} = x_{\text{actual}} + v \cdot dt$$

Esto significa:

- Tomamos la posición actual.
- Calculamos cuánto debe avanzar.
- Sumamos ese avance a la posición.

Esta es una forma básica del método de Euler.

Movimiento sin dt

El movimiento incorrecto usa una constante fija:

$$X_{\text{nuevo}} = X_{\text{actual}} + v \cdot c$$

En el código:

$$c = 0,016$$

Esto supone artificialmente que todos los frames duran:

0,016 segundos

Esto solo se parece a 60 FPS.

Problema de no usar dt

Si el programa corre a 30 FPS, hay menos actualizaciones por segundo.

Si corre a 120 FPS, hay más actualizaciones por segundo.

Usando una constante fija:

$$\text{distancia por segundo} = \text{actualizaciones} \times v \cdot c$$

Por eso el movimiento depende de los FPS.

Ejemplo sin dt

Supongamos:

$$v = 200, \quad c = 0,016$$

Avance por frame:

$$\Delta x = 200(0,016) = 3,2$$

A 30 FPS:

$$30(3,2) = 96$$

A 120 FPS:

$$120(3,2) = 384$$

El objeto no se mueve igual.

Movimiento con dt

Con dt , cada frame usa su duración real:

$$x_{\text{nuevo}} = x_{\text{actual}} + v \cdot dt$$

A 30 FPS:

$$30 \cdot 200 \cdot 0,0333 \approx 200$$

A 120 FPS:

$$120 \cdot 200 \cdot 0,0083 \approx 200$$

El movimiento respeta el tiempo real.

Comparación conceptual

Caso	Fórmula	Consecuencia
Sin dt	$x = x + v \cdot c$	Depende de FPS
Con dt	$x = x + v \cdot dt$	Estable

dt corrige la duración real de cada frame.

Integración numérica: idea básica

El modelo continuo es:

$$\frac{dx}{dt} = v$$

Pero el computador no calcula todos los tiempos.

Calcula una secuencia discreta:

$$t_0, t_1, t_2, \dots, t_n$$

La separación entre dos tiempos es:

$$\Delta t = t_{n+1} - t_n$$

Lectura del programa

El programa compara dos modelos:

- Círculo rojo: usa una constante fija.
- Círculo azul: usa el dt real.

Al cambiar FPS:

- El rojo cambia su rapidez visible.
- El azul conserva su rapidez.

Conclusión

Ideas principales:

- FPS indica cuántos frames se dibujan por segundo.
- dt mide cuánto duró el último frame.
- Sin dt , el movimiento depende de los FPS.
- Con dt , el movimiento respeta el tiempo real.
- La fórmula $x = x + vdt$ es integración numérica.
- La integral continua produce $x(t) = x_0 + vt$.